Working with Navigation Bar

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}
```

Tells the Flutter framework to inflate the MyApp widget and make it the root of the widget tree displayed on the screen.

```dart
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: "Navigation Activity",
      theme: ThemeData(colorScheme: ColorScheme.light()),
      home: HomeScreen(),
    );
  }
}
```

```dart
class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  // The data to be searched (const is needed because HomeScreen has a const constructor)
  final List<String> coffees = const [
    "Americano",
    "Cappuccino",
    "Espresso",
    "Flat White",
    "Latte",
    "Macchiato",
    "Mocha",
  ];
```

coffees list: *A static list of string names for different coffee types.*

```dart
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Search")),
      body: Column(
        children: [
          // Search bar (not working yet)
          Padding(
            padding: EdgeInsets.all(16),
            child: TextField(
              decoration: InputDecoration(
                hintText: "Search coffee",
                prefixIcon: Icon(Icons.search),
                filled: true,
                fillColor: Color(0xFFEEEEEE),
```

```
                border: OutlineInputBorder(
                  borderRadius: BorderRadius.circular(30),
                  borderSide: BorderSide.none,
                ),
              ),
            ),
          ),
```

***TextField**: Styled with a grey background (fillColor), a search icon prefix, and rounded borders, but missing an onChanged function.*

```
          // Shows every coffee for now
          Expanded(
            child: ListView.builder(
              itemCount: coffees.length,
              itemBuilder: (context, index) {
                return ListTile(
                  leading: Icon(Icons.coffee),
                  title: Text(coffees[index]),
                );
              },
            ),
          ),
        ],
      ),
    );
  }
}
```

***ListView.builder**: Renders every single item in the coffees list regardless of what is typed into the search box..*

Next we are going to make our search Work here through the use of:

1. **Stateful Widget**
2. **Filtering Logic in the Build Method**
3. **Inetractive Search Bar and Dynamic Result**
   a) **OnChanged**
   b) **Ternary conditional Operator**

```
// Changed to StatefulWidget so the screen can change when a tab is tapped
class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  int _selectedIndex = 0; // which bottom bar tab is active
```

- ***StatefulWidget**: Enables the screen to manage state variables that change over time.*
- ***_searchText**: A state variable that records the current text entered by the user (starts as an empty string "").*

```
  @override
  Widget build(BuildContext context) {
    // Keep only the coffees that contain the typed text
    final results = coffees
```

```
        .where((coffee) =>
            coffee.toLowerCase().contains(_searchText.toLowerCase()))
        .toList();
```

- ***.where**(...): Filters the main coffees list. It checks if each coffee name contains the _searchText.*
- ***.toLowerCase**(): Ensures the search is case-insensitive (so typing "latte" or "Latte" both match "Latte").*
- ***.toList**(): Converts the filtered result back into a standard List<String>.*

```
  return Scaffold(
      appBar: AppBar(title: Text("Search")),
      body: Column(
        children: [
          // Search bar
          Padding(
            padding: EdgeInsets.all(16),
            child: TextField(
              onChanged: (value) {
                setState(() {
                  _searchText = value;
                });
              },
              decoration: InputDecoration(
                hintText: "Search coffee",
                prefixIcon: Icon(Icons.search),
                filled: true,
                fillColor: Color(0xFFEEEEEE),
                border: OutlineInputBorder(
                  borderRadius: BorderRadius.circular(30),
                  borderSide: BorderSide.none,
                ),
              ),
            ),
          ),

          // Show the results, or a message when nothing matches
          Expanded(
            child: results.isEmpty
                ? Center(child: Text("No coffee found"))
                : ListView.builder(
                    itemCount: results.length,
                    itemBuilder: (context, index) {
                      return ListTile(
                        leading: Icon(Icons.coffee),
                        title: Text(results[index]),
                      );
                    },
                  ),
          ),
        ],
      ),
    );
  }
}
```

- ***onChanged: (value) { setState(() { _searchText = value; }); }:*** *Captures every keystroke in the text field, updates _searchText, and calls setState to trigger an immediate screen rebuild.*
- ***results.isEmpty ? ... : ...: A ternary conditional operator:***
  - *If no matches are found, it displays a center text message saying* ***"No coffee found".***
  - *If matches exist, it uses* ***ListView.builder*** *to render only the filtered results.*